

Redes Recorrentes: RNNs e LSTMs

n:

ar 2021

er P' P' P'

Parte 1 — O Problema das Sequências

Roteiro da aula

Parte 1 — O Problema das Sequências Por que redes densas e CNNs não bastam

Parte 2 — Redes Recorrentes (RNN) Estado oculto, equação recorrente, desenrolamento

Parte 3 — Treinando RNNs BPTT, vanishing gradient, gradient clipping

Parte 4 — LSTM Célula de memória, 4 portas, intuição

Parte 5 — GRU e Arquiteturas Práticas GRU, bidirecional, empilhamento, dropout

Parte 6 — Código e Aplicações PyTorch e Keras; classificação e seq2seq

Dados sequenciais estão em todo lugar

Texto / NLP

"O **gato** que **caçou** o rato **dormia** agora."

A palavra "dormia" só faz sentido depois de entender "gato" três tokens atrás.

Séries temporais

Temperatura, preço de ações, sinal de EEG.

O valor em t depende dos valores em $t-1, t-2, \dots$

Áudio / fala

O fonema atual depende do contexto fonético anterior.

Propriedade-chave: a **ordem importa** e o comprimento das sequências é **variável**.

Isso quebra as premissas de redes densas (entrada de tamanho fixo, sem noção de posição) e CNNs (campo receptivo local fixo).

Por que redes densas falham em sequências

Abordagem ingênua: concatenar todos os tokens e passar para uma rede densa.

```
"o gato caçou o rato"  
→ [emb_o || emb_gato || emb_caçou || emb_o || emb_rato]  
→ Dense(5 × d → hidden) → Dense(hidden → saída)
```

Problema 1 — Comprimento variável

Frases de 3, 10, 100 tokens → entradas de tamanhos diferentes.

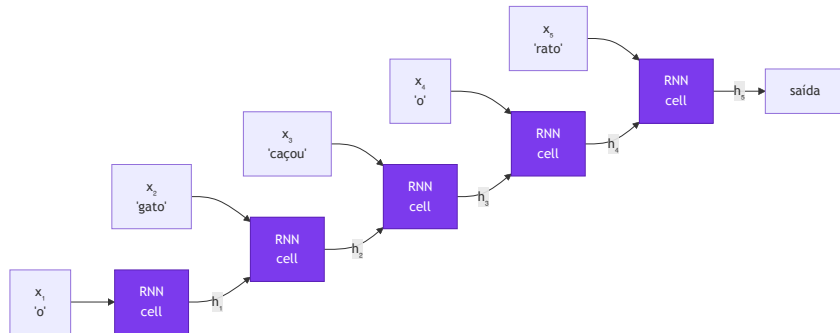
Problema 2 — Sem compartilhamento de pesos

O detector de "gato" na posição 2 é um conjunto diferente de pesos do que na posição 7. Não generaliza.

Problema 3 — Não escala

O que precisamos

- ✓ Processar tokens **um de cada vez**, em ordem
- ✓ Manter um **estado interno** que acumula informação do passado
- ✓ **Compartilhar pesos** entre todas as posições
- ✓ Funcionar com sequências de **tamanho arbitrário**



Parte 2 — Redes Neurais Recorrentes (RNN)

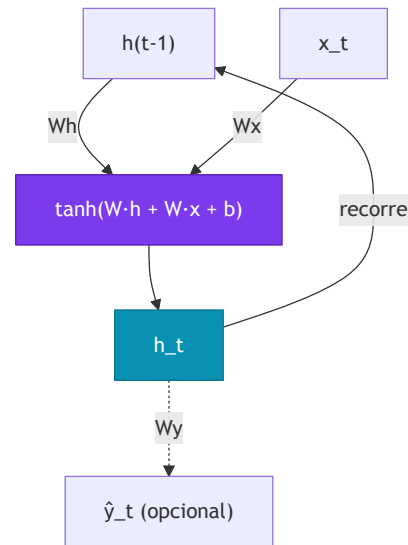
A célula RNN

Ideia central: uma célula que processa o token atual $\mathbf{x}_t$ e o estado oculto anterior $\mathbf{h}_{t-1}$, produzindo um novo estado $\mathbf{h}_t$.

$$\mathbf{h}_t = \tanh(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + \mathbf{b})$$

- $\mathbf{x}_t \in \mathbb{R}^d$ — embedding do token t
- $\mathbf{h}_t \in \mathbb{R}^h$ — estado oculto no passo t (a "memória")
- $W_x \in \mathbb{R}^{h \times d}$ — projeta a entrada
- $W_h \in \mathbb{R}^{h \times h}$ — projeta o estado anterior
- $\mathbf{b} \in \mathbb{R}^h$ — bias
- $\tanh$ — ativa e mantém valores em $[-1, 1]$

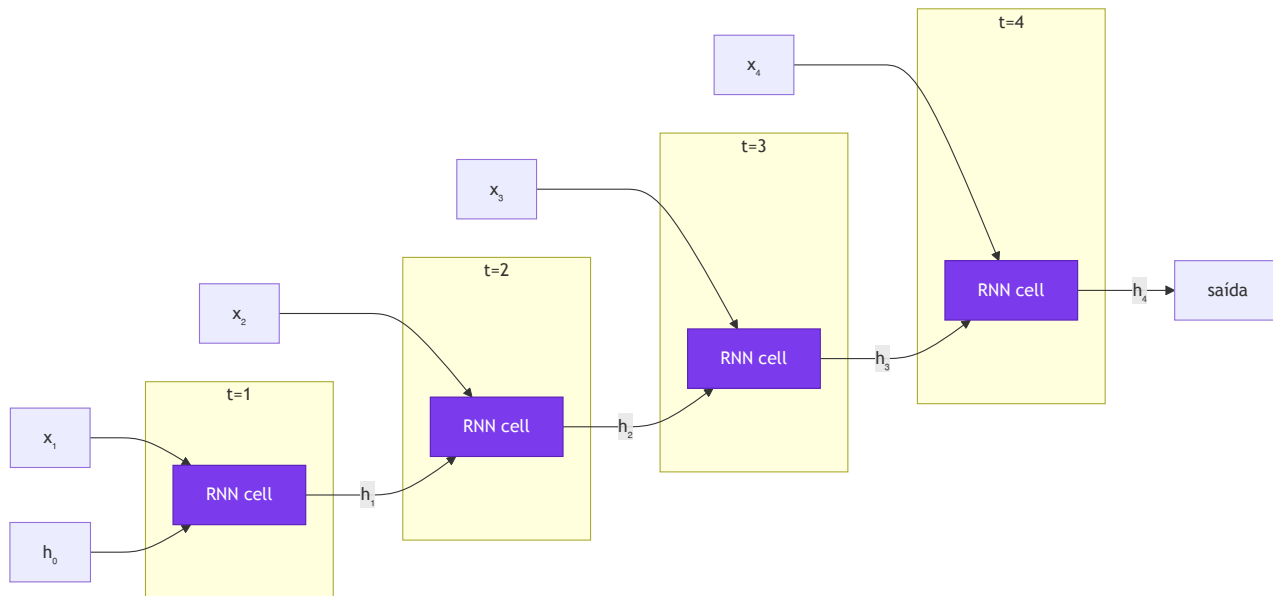
Os mesmos pesos $W_h, W_x, \mathbf{b}$ são usados em **todos** os passos de tempo — isso é o compartilhamento de pesos.



A seta de loop ($\mathbf{h}_t$ volta para $\mathbf{h}_{t-1}$ no próximo passo) é o que define a recorrência.

Desenrolando no tempo (*unrolling*)

A mesma célula repetida para cada passo de tempo:



Pesos compartilhados

$W_h, W_x, \mathbf{b}$ são os **mesmos** em todos os

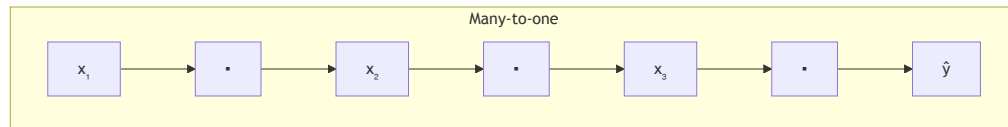
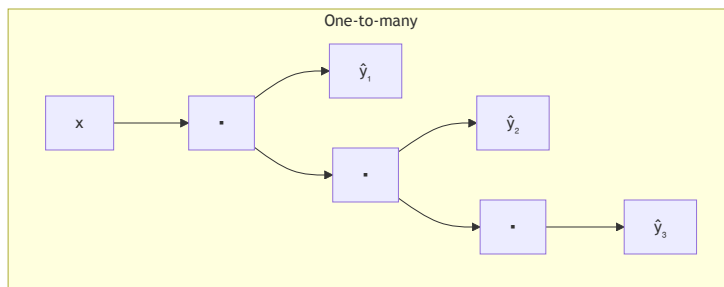
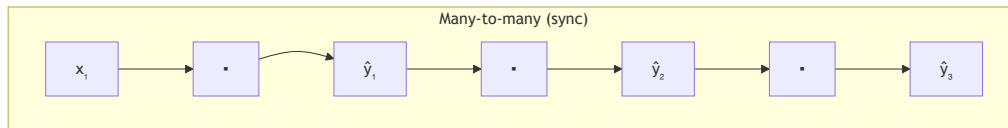
Estado inicial

$\mathbf{h}_0$ inicializado com zeros. Cada sequência no batch tem seu próprio $\mathbf{h}_0$.

Saída

$\mathbf{h}_T$ (último estado) para classificação; todos

Topologias de entrada/saída



Many-to-one — Sequência → rótulo

Ex: sentimento, classificação de texto

One-to-many — Vetor → sequência

Ex: *image captioning*

Many-to-many — Seq → seq

Ex: NER (mesmo comprimento), seq2seq (comprimento diferente)

Parte 3 — Treinando RNNs

Backpropagation Through Time (BPTT)

O gradiente da loss precisa fluir de volta por cada passo de tempo — através de cada aplicação de W_h .

$$\frac{\partial \mathcal{L}}{\partial W_h} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t}{\partial W_h}$$
$$\frac{\partial \mathcal{L}_t}{\partial W_h} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \cdot \prod_{k=t}^{T-1} \frac{\partial \mathbf{h}_{k+1}}{\partial \mathbf{h}_k}$$

$\text{diag}(\tanh'(\cdot))$ é a matriz diagonal das derivadas do tanh no passo k — cada elemento é $1 - h_k^2 \in (0, 1)$. Multiplicar por $W_h^\top$ a cada passo encolhe ou explode o gradiente.

O problema:

$$\prod_{k=t}^{T-1} \frac{\partial \mathbf{h}_{k+1}}{\partial \mathbf{h}_k} = \prod_{k=t}^{T-1} W_h^\top \cdot \text{diag}(\tanh'(\cdot))$$

Vanishing gradient

Se $\|W_h\| < 1$ ou derivadas de $\tanh$ são pequenas, o produto encolhe **exponencialmente** → gradiente ≈ 0 para dependências longas

Exploding gradient

Se $\|W_h\| > 1$, o produto cresce **exponencialmente** → NaN / divergência no treino

Vanishing gradient — intuição

Imagine uma cadeia de multiplicações com fator $\gamma < 1$ a cada passo:

```
T = 10 passos,  $\gamma = 0.9$ :  
gradiente  $\leftarrow 0.9^{10} \approx 0.35$  (ainda ok)  
  
T = 50 passos,  $\gamma = 0.9$ :  
gradiente  $\leftarrow 0.9^{50} \approx 0.005$  (quase zero)  
  
T = 100 passos,  $\gamma = 0.9$ :  
gradiente  $\leftarrow 0.9^{100} \approx 2 \times 10^{-5}$  (efetivamente zero)
```

A rede "esquece" dependências acima de ~10-20 passos.

Consequência prática

❌ RNNs vanilla falham em dependências longas (tradução, documentos)

Solução parcial: gradient clipping

```
# PyTorch — clipar gradientes antes do update  
torch.nn.utils.clip_grad_norm_  
    (model.parameters(), max_norm=1.0  
)
```

✅ Resolve o *exploding gradient* · ❌ não resolve o vanishing → precisa de LSTM

Parte 4 — Long Short-Term Memory (LSTM)

Motivação — uma memória explícita

Hochreiter & Schmidhuber (1997)

A ideia central: separar a "memória de longo prazo" do "estado de trabalho".

RNN vanilla:

Um único vetor $\mathbf{h}_t$ deve fazer tudo — acumular história, filtrar ruído, carregar informação útil.

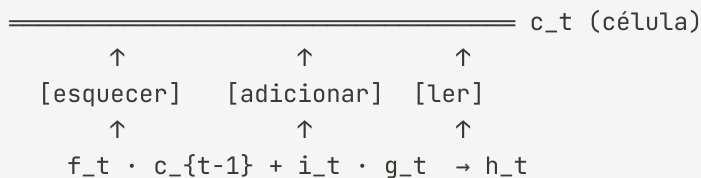
LSTM:

Dois vetores distintos:

- $\mathbf{c}_t$ — **célula de memória** (longo prazo, fluxo suave)
- $\mathbf{h}_t$ — **estado oculto** (curto prazo, saída filtrada)

E três **portas** (gates) que controlam o fluxo de informação.

A metáfora da esteira



- $\mathbf{f}_t$ (*forget gate*) — quanto de $\mathbf{c}_t$ preservar (0 = apaga, 1 = mantém)
- $\mathbf{i}_t$ (*input gate*) — quanto da proposta $\mathbf{g}_t$ deve ser escrita na célula
- $\mathbf{g}_t$ (*candidato*) — proposta de novo conteúdo, gerada por $\tanh$
- $\mathbf{h}_t$ — saída filtrada pelo *output gate* a partir de $\mathbf{c}_t$

LSTM — Equação 1: Forget Gate

$$\mathbf{f}_t = \sigma(W_f [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f)$$

"O filtro do passado"

- $\sigma(\cdot) \rightarrow$ saída em **(0, 1)** por dimensão independentemente
- $\mathbf{f}_t \approx \mathbf{0} \rightarrow$ apaga essa dimensão de $c(t-1)$
- $\mathbf{f}_t \approx \mathbf{1} \rightarrow$ preserva essa dimensão intacta
- Aprende o que descartar a partir de $h(t-1)$ e x_t

Exemplo: ao processar "Pedro" após "João foi ao mercado", $f_t \approx 0$ na dimensão do sujeito ativo **apaga "João"** para dar lugar ao novo referente.

Gradiente: $\partial c_t / \partial c(t-1) = f_t$. Quando $f_t \approx 1$, o gradiente volta **sem atenuação** — solução central para o *vanishing gradient*.

LSTM — Equação 2: Input Gate + Candidato

$$\mathbf{i}_t = \sigma(W_i [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i) \quad \tilde{\mathbf{c}}_t = \tanh(W_c [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_c)$$

"O quê" e "quanto" escrever

Dois papéis distintos que trabalham juntos:

- $\hat{\mathbf{c}}_t$ (tanh) — propõe o *conteúdo*: valores em $(-1, 1)$, codifica direção/polaridade
- $\mathbf{i}_t$ (σ) — a *porteira*: decide quanto de $\hat{\mathbf{c}}_t$ realmente entra na célula
- Produto: $\mathbf{i}_t \odot \hat{\mathbf{c}}_t$ = nova informação filtrada

Exemplo: ao processar "Pedro", $\hat{\mathbf{c}}_t$ codifica as propriedades do novo sujeito; $\mathbf{i}_t$ decide em quais dimensões da célula gravá-las.

Por que tanh para $\hat{\mathbf{c}}_t$? A memória precisa representar *direção* — incrementar ou decrementar uma feature. σ só dá valores positivos; tanh permite codificar polaridade.

LSTM — Equação 3: Atualização da Célula

$$\mathbf{c}_t = \underbrace{\mathbf{f}_t \odot \mathbf{c}_{t-1}}_{\text{apaga o velho}} + \underbrace{\mathbf{i}_t \odot \tilde{\mathbf{c}}_t}_{\text{adiciona o novo}}$$

A "esteira de memória"

A célula $\mathbf{c}_t$ é uma **via aditiva** que transporta informação ao longo do tempo:

- $\mathbf{f}_t \odot \mathbf{c}_{t-1} \rightarrow$ mantém o que ainda importa
- $\mathbf{i}_t \odot \tilde{\mathbf{c}}_t \rightarrow$ acrescenta o novo conteúdo
- $\odot =$ produto de Hadamard (elemento a elemento)

Casos extremos: $\mathbf{f}_t \approx 1, \mathbf{i}_t \approx 0 \rightarrow$ célula **congelada** · $\mathbf{f}_t \approx 0, \mathbf{i}_t \approx 1 \rightarrow$ célula **resetada**

Por que a soma resolve o vanishing gradient?

Na RNN vanilla o gradiente flui por $W_h \cdot \tanh'(\cdot)$ e encolhe exponencialmente a cada passo.

Na LSTM: $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \mathbf{f}_t$. Quando $\mathbf{f}_t \approx 1$ o gradiente volta **diretamente**, sem passar por não-linearidades — a adição cria um "atalho" na backpropagation.

LSTM — Equação 4: Output Gate

$$\mathbf{o}_t = \sigma(W_o [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o) \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

"Memória privada vs. saída pública"

- $\mathbf{c}_t$ = memória interna (pode guardar muito mais do que é exposto)
- $\mathbf{o}_t$ (σ) = decide quais dimensões de $\mathbf{c}_t$ "vazam" para $\mathbf{h}_t$
- $\tanh(\mathbf{c}_t)$ = normaliza a célula para $(-1, 1)$
- $\mathbf{h}_t$ = saída visível — vai para a próxima camada e para $t+1$

Exemplo: em tradução, $\mathbf{c}_t$ guarda o gênero de um substantivo por vários passos. O output gate o expõe apenas quando o modelo vai gerar um adjetivo que precisa de concordância.

Síntese: forget → apagar $\mathbf{c}(t-1)$ · input → escrever em $\mathbf{c}_t$ · output → expor $\mathbf{h}_t$

Intuição dos gates — passo a passo

Exemplo: "Maria foi ao mercado. Ela comprou maçãs." — a rede precisa saber que "Ela" = "Maria".

Forget gate

Ao processar "Ela", a rede pode usar $\mathbf{f}_t \approx 1$ para **manter** "Maria" na célula, ou $\mathbf{f}_t \approx 0$ para **apagar** informações irrelevantes de passos anteriores.

$$\mathbf{c}_t \leftarrow \mathbf{f}_t \odot \mathbf{c}_{t-1}$$

Input gate

Ao ver "Ela", $\mathbf{i}_t$ decide quanto do candidato $\tilde{\mathbf{c}}_t$ (que codifica o pronome e seu possível referente) deve ser **gravado** na célula.

$$\mathbf{c}_t \mathrel{+}= \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

Output gate

Ao gerar a próxima palavra, $\mathbf{o}_t$ controla **quanta** da memória de "Maria" deve ser passada para $\mathbf{h}_t$ e, portanto, influenciar a predição.

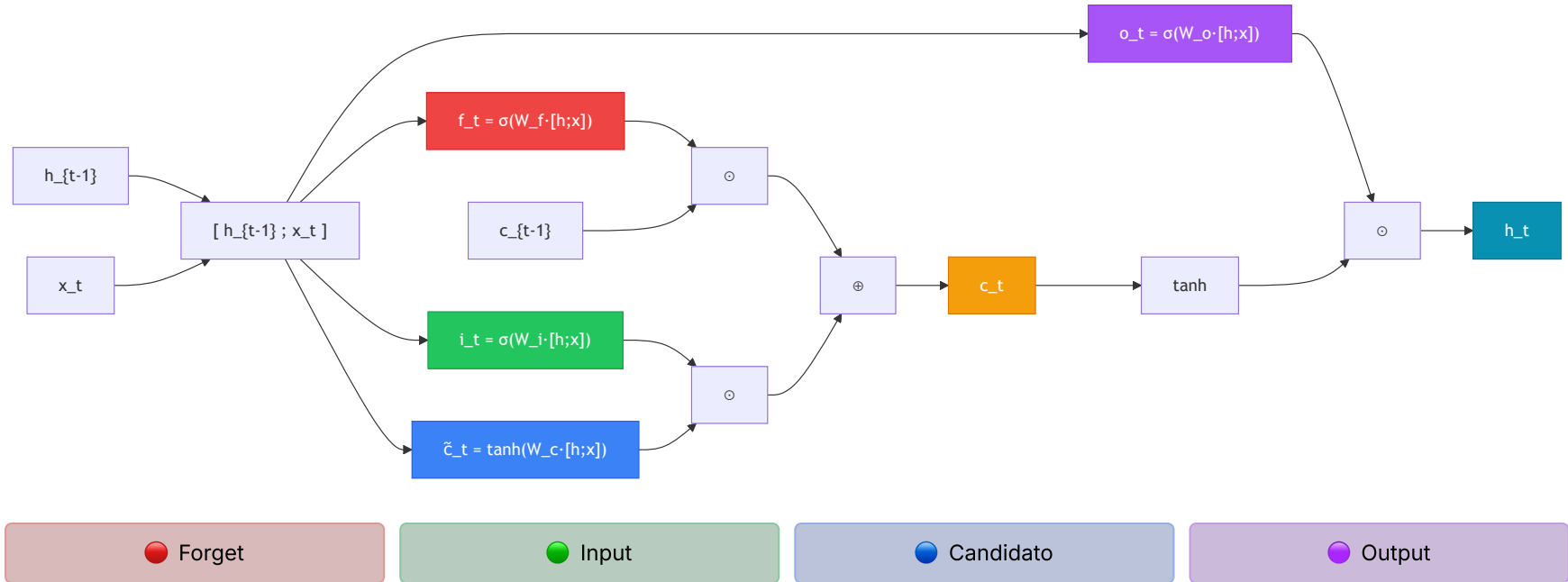
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Por que a LSTM resolve o vanishing gradient?

O caminho de gradiente pela célula $\mathbf{c}$ é **aditivo**: $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \dots$

A adição não multiplica o gradiente — ele pode fluir de volta por muitos passos sem encolher exponencialmente (desde que $\mathbf{f}_t \approx 1$).

Diagrama completo da célula LSTM



Parte 5 — GRU e Arquiteturas Práticas

GRU — Gated Recurrent Unit

Cho et al. (2014) — versão simplificada da LSTM com apenas 2 gates e sem célula separada.

Porta de reset (*reset gate*)

$$\mathbf{r}_t = \sigma(W_r [\mathbf{h}_{t-1}; \mathbf{x}_t])$$

Controla quanto do estado anterior influencia o candidato.

Porta de atualização (*update gate*)

$$\mathbf{z}_t = \sigma(W_z [\mathbf{h}_{t-1}; \mathbf{x}_t])$$

Candidato + atualização final

$$\tilde{\mathbf{h}}_t = \tanh(W [\mathbf{r}_t \odot \mathbf{h}_{t-1}; \mathbf{x}_t])$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

GRU vs LSTM

	LSTM	GRU
Gates	3 (f, i, o)	2 (r, z)
Estado	$\mathbf{h}_t + \mathbf{c}_t$	só $\mathbf{h}_t$
Parâmetros	Mais	Menos
Treinamento	Mais lento	Mais rápido
Desempenho	Melhor em seq. muito longas	Comparável na maioria

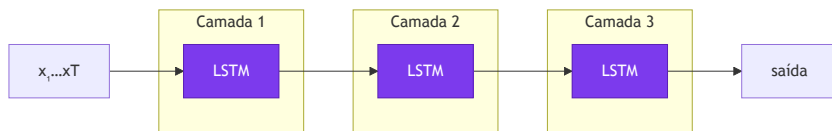
Regra prática: comece com GRU; use LSTM para sequências muito longas ou quando precisar de memória mais granular.

RNN Bidireccional

RNNs empilhadas e Dropout

Empilhamento (*stacking*)

A saída de uma camada RNN alimenta a próxima — cada camada aprende representações mais abstratas.



2–4 camadas são comuns; mais do que isso raramente ajuda e aumenta o risco de overfitting.

Dropout em RNNs

Dropout padrão entre camadas funciona, mas **não** dentro da célula recorrente (destruiria a memória).

Dropout variacional (Gal & Ghahramani, 2016): aplica a **mesma máscara** em todos os passos de tempo.

```
# PyTorch – dropout entre camadas LSTM
nn.LSTM(
    input_size=128,
    hidden_size=256,
    num_layers=3,
    dropout=0.3,      # aplicado entre camadas
    bidirectional=False
)
```

O argumento `dropout` do `nn.LSTM` só é aplicado entre as camadas intermediárias — a última camada não tem dropout automático.

Parte 6 — Código e Aplicações

nn.LSTM e nn.GRU — PyTorch

```
import torch.nn as nn

x = torch.randn(2, 10, 64) # (batch, seq, feat)

lstm = nn.LSTM(input_size=64, hidden_size=128,
               num_layers=2, batch_first=True,
               dropout=0.2, bidirectional=False)

h0 = torch.zeros(2, 2, 128) # (layers, batch, hidden)
c0 = torch.zeros(2, 2, 128)

output, (hn, cn) = lstm(x, (h0, c0))
# output: (2, 10, 128) - todos os h_t
# hn/cn: (2, 2, 128) - último estado de cada camada
```

```
# GRU - mesma interface, sem c
gru = nn.GRU(input_size=64, hidden_size=128,
             num_layers=2, batch_first=True,
             bidirectional=True) # saída ×2

output, hn = gru(x)
# output: (2, 10, 256) bidirecional → ×2
# hn: (4, 2, 128) 2 layers × 2 direções

last_hidden = output[:, -1, :] # último token
mean_hidden = output.mean(dim=1) # média
```

Use sempre `batch_first=True` — formato `(batch, seq, feat)`, consistente com Keras.

Classificação de sentimento — PyTorch

```
class SentimentLSTM(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers, num_classes):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers,
                             batch_first=True, dropout=0.3)
        self.dropout = nn.Dropout(0.3)
        self.fc = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        # x: (batch, seq_len)
        emb = self.embed(x)           # (batch, seq_len, embed_dim)
        out, (hn, _) = self.lstm(emb) # out: (batch, seq_len, hidden)
        # Usa o último estado oculto da última camada
        last = hn[-1]                 # (batch, hidden_dim)
        last = self.dropout(last)
        return self.fc(last)          # (batch, num_classes)

model = SentimentLSTM(10_000, 128, 256, 2, 2)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for tokens, labels in train_loader:
    logits = model(tokens)
    loss = criterion(logits, labels)
```

LSTM em Keras

```
import tensorflow as tf
from tensorflow import keras

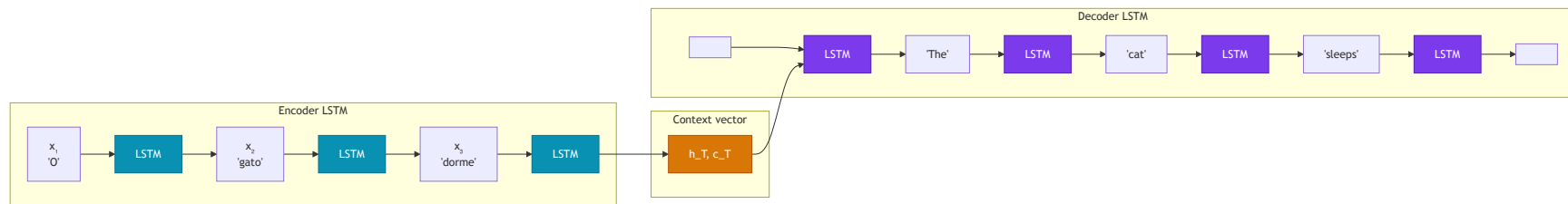
# Pipeline completo
vectorizer = keras.layers.TextVectorization(
    max_tokens=10_000,
    output_sequence_length=128
)
vectorizer.adapt(train_texts)

model = keras.Sequential([
    keras.Input(shape=(1,)), dtype='string'),
    vectorizer,
    keras.layers.Embedding(10_000, 128, mask_zero=True),
    # LSTM bidirecional empilhado
    keras.layers.Bidirectional(
        keras.layers.LSTM(128, return_sequences=True,
                           dropout=0.2, recurrent_dropout=0
        ),
    ),
    keras.layers.Bidirectional(
        keras.layers.LSTM(64, dropout=0.2)
    ),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dropout(0.3),
```

```
model.compile(
    loss='binary_crossentropy',
    optimizer=keras.optimizers.Adam(1e-3),
    metrics=['accuracy']
)

# Treinar
history = model.fit(
    train_texts, train_labels,
    validation_split=0.2,
    epochs=10, batch_size=64
)
```

Aplicação: sequência de texto a tradução (seq2seq)



Encoder: processa a sequência de entrada, compressa em vetor de contexto (h_T, c_T) .

Decoder: inicializado com (h_T, c_T) , gera a saída token por token de forma autorregressiva.

Limitação do seq2seq puro: o vetor de contexto é um gargalo — comprimir uma sequência inteira em um único vetor perde informação. A solução é

Comparativo: RNN, LSTM, GRU

	RNN vanilla	LSTM	GRU
Memória	Estado $\mathbf{h}_t$	$\mathbf{h}_t + \mathbf{c}_t$	só $\mathbf{h}_t$
Gates	—	3 (f, i, o)	2 (r, z)
Parâmetros	$4\times$ (vanilla)	$4\times$ mais que RNN	$3\times$ mais que RNN
Dependências longas	❌ Vanishing	✅ Bom	✅ Bom
Velocidade	Rápido	Mais lento	Intermediário
Uso típico	Baseline rápido	Padrão em NLP até 2018	Alternativa eficiente

2017–2018: LSTMs dominavam tradução, sumarização e reconhecimento de fala. Com os **Transformers** (Vaswani et al., 2017), foram gradualmente

Recapitulando

O problema

- Dados sequenciais têm ordem e tamanho variável
- Redes densas: sem ordem, sem compartilhamento
- CNNs: campo receptivo limitado

RNN vanilla

- $\mathbf{h}_t = \tanh(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t)$
- Pesos compartilhados em todos os passos
- Sofre de vanishing gradient

BPTT e gradientes

- Gradiente flui de volta por multiplicações encadeadas
- Vanishing: $\gamma^T \rightarrow 0$ com $\gamma < 1$
- Exploding: gradient clipping

LSTM

- Célula $\mathbf{c}_t$ + gates f , i , o
- Caminho aditivo: resolve vanishing
- $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$

GRU e práticas

- 2 gates, mais eficiente que LSTM
- Bidirecional: contexto passado + futuro
- Empilhamento + dropout variacional

Próxima aula

- Gargalo do vetor de contexto
- Mecanismo de atenção
- Arquitetura Transformer completa

Próxima aula

Aula 6 — Transformers

- Limitação do seq2seq como motivação
- Mecanismo de atenção (*self-attention*)
- *Scaled Dot-Product Attention* e Multi-Head
- Positional encoding
- Arquitetura completa (encoder + decoder)
- BERT e GPT: fine-tuning e geração

Para esta semana

- Notebook `lec05-rnn.ipynb` :
 - Classificação de sentimento com LSTM (IMDB)
 - Comparação RNN vs LSTM vs GRU
 - Curvas de aprendizado e análise de gradientes
 - Visualização dos estados ocultos com t-SNE

Obrigado! Perguntas?

CCM-109 · Deep Learning · UFABC